

计算机系统基础

姚文斌

计算机系统结构中心

13691048256

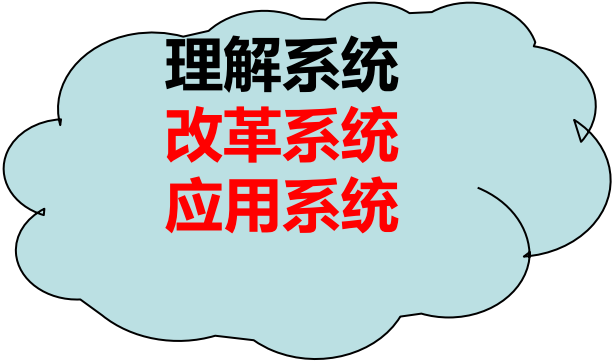
yaowenbin_cdc@163.com

课程目的

- 从程序员角度出发，理解高级语言中的数据、语句和函数调用等在计算机系统中的实现细节
- 建立C语言程序、汇编语言、编译器、链接器等之间的关联关系
- 初步理解基本输入输出控制方式、I/O函数的用法和设备驱动程序基本原理
- 建立起层次型计算机系统概念，增强在程序设计、程序调试等方面的动手能力，为后续课程打下坚实基础

系统能力基于对系统的理解

- 了解计算机系统整体概念，理解计算机系统层次结构
- 理解高级语言程序、ISA、编译/链接、OS、硬件等之间的关系
 - 高级语言语句与具体指令的对应关系
 - 变量（常量）如何表示和存放
 - 数组、指针等如何在指令级进行访问操作
 - 堆/栈的结构和动态存储分配机制
 - 程序中的I/O操作和涉及到的系统调用过程
 -
- 理解指令在计算机硬件上的执行过程
 - 算术逻辑运算部件以及运算指令执行过程
 - I/O结构（I/O外设和接口、BUS、网络等）以及I/O过程
 -
-



理解系统
改革系统
应用系统

课程内容概要

```
/*---sum.c---*/
```

```
int sum(int a[ ], unsigned len)
```

```
{
```

```
    int i    sum = 0;
```

```
    for (i = 0; i <= len-1; i++)
```

```
        sum += a[i];
```

```
    return sum;
```

```
}
```

```
/*---main.c---*/
```

```
int main()
```

```
{
```

```
    int a[1]={100};
```

```
    int sum;
```

```
    sum=sum(a,0);
```

```
    printf("%d",sum);
```

```
}
```

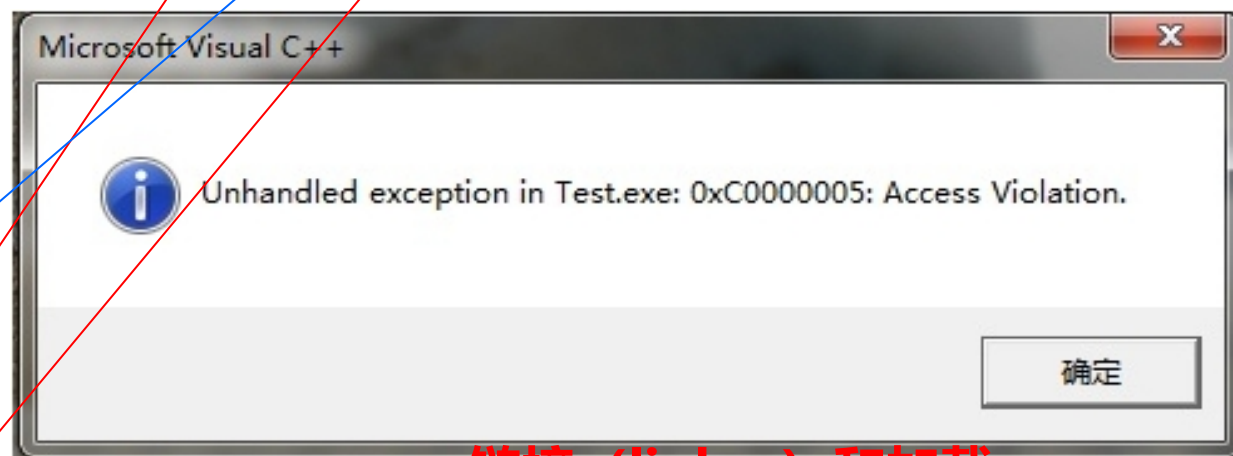
数据的表示

数据的运算

各类语句的转换与表示(指令)

各类复杂数据类型的转换表示

过程（函数）调用的转换表示



链接 (linker) 和加载

程序执行 (存储器访问)

异常和中断处理

输入输出(I/O)

课程内容概要

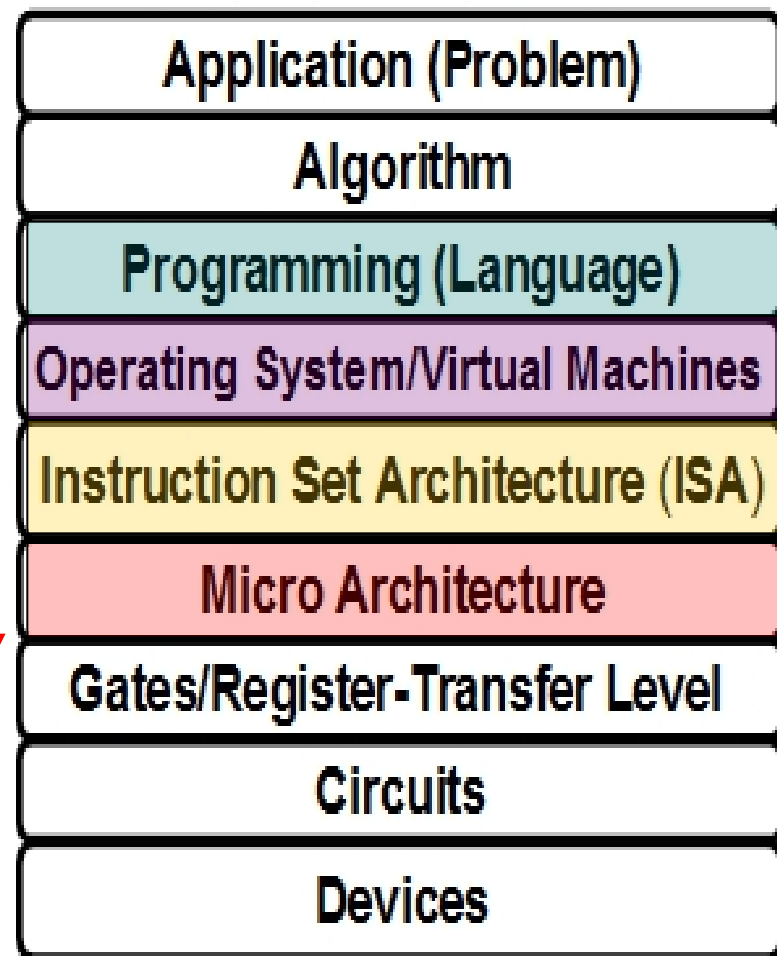
- 使学生清楚理解：

计算机是如何生成和运行可执行文件的！

“问题求解”课程解
决应用→算法（数据
结构）→编程层

- 重点在高级语言以下各抽象层

- C语言程序设计层
 - 数据的机器级表示、运算
 - 语句和过程调用的机器级表示
- 指令集体系结构（ISA）和汇编层
 - 指令系统、机器代码、汇编语言
- 微体系结构及硬件层
 - CPU的通用结构
 - 层次结构存储系统
- 操作系统、编译和链接的部分内容



三个主题:

- **表示 (Representation)**
 - 不同数据类型（带符号整数、无符号整数、浮点数、数组、结构等）在寄存器或存储器中如何表示和存储？
 - 指令如何表示和编码（译码）？
 - 存储地址（指针）如何表示以及如何生成复杂数据结构中数据元素的地址？
- **转换 (Translation)**
 - 高级语言程序对应的机器级代码是怎样的？
- **执行控制流 (Control flow)**
 - 计算机能理解的“程序”是如何组织和控制的？
 - 如何在计算机中组织多个程序的并发执行？
 - 逻辑控制流中的异常事件及其处理
 - I/O操作的执行控制流（用户态→内核态）

前导知识：C语言程序设计+计算机导论

内容组织：两大部分

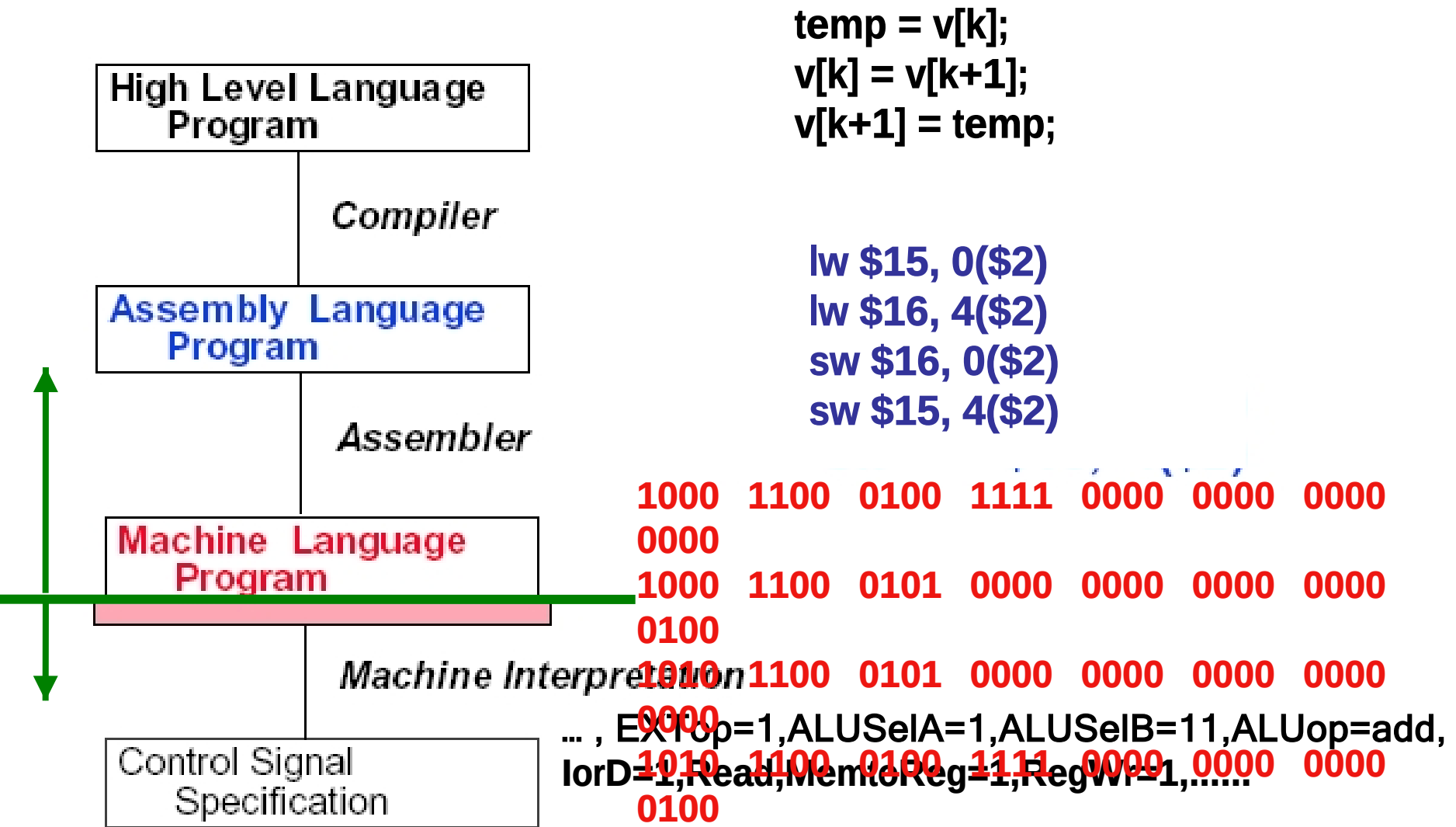
第一部分 系统概述和可执行文件的生成（表示和转换）

- 计算机系统概述
- 数据的机器级表示与处理
- 程序的转换及机器级表示
- 程序的链接

第二部分 可执行文件的运行（执行控制流）

- 程序的执行
- 异常控制流
- I/O操作的实现

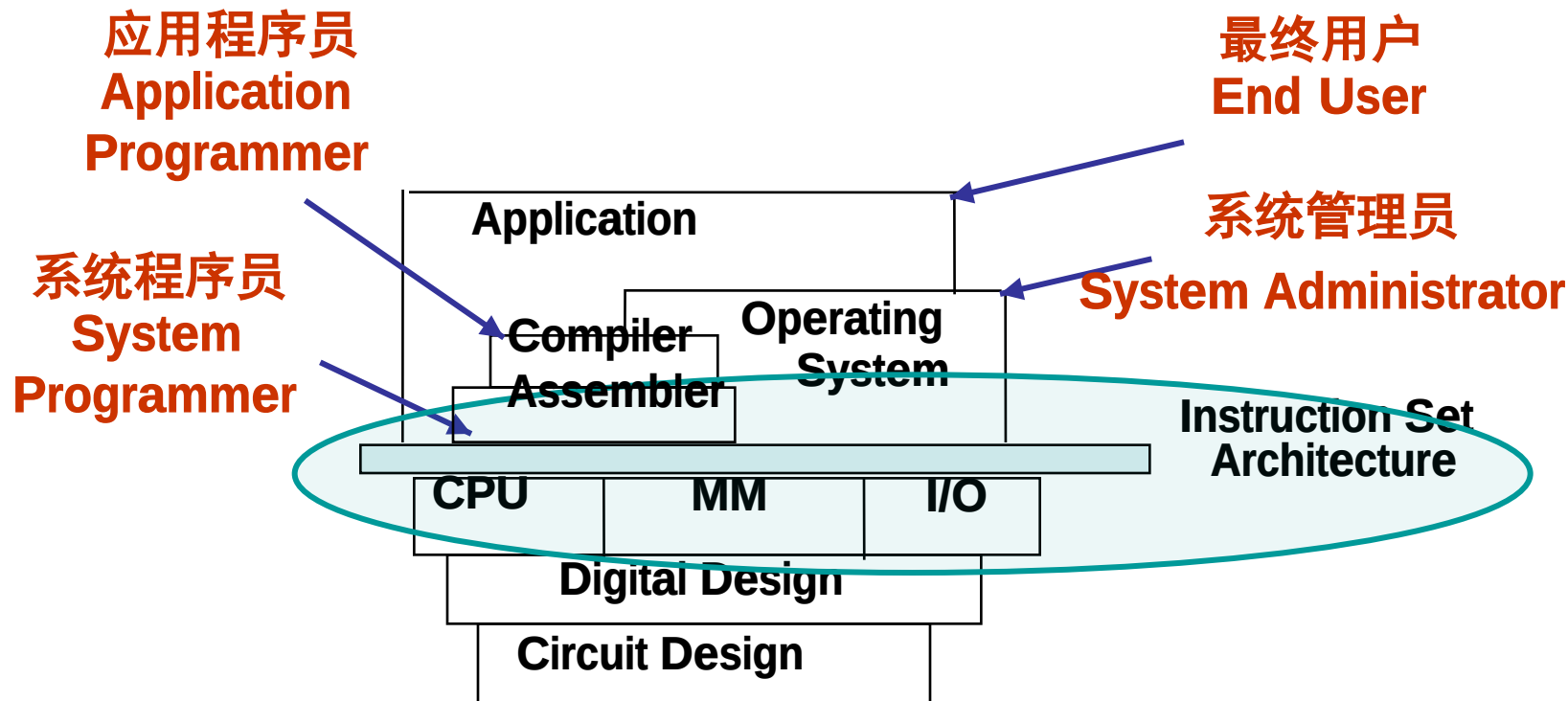
Hardware/Software Interface



- **System software(系统软件)** - 简化编程过程, 并使硬件资源被有效利用
 - 操作系统 (Operating System) : 硬件资源管理, 用户接口
 - 语言处理系统: 翻译程序+ Linker, Debug, etc ...
 - 翻译程序(Translator)有三类:
 - 汇编程序(Assembler):** 汇编语言源程序→机器语言目标程序
 - 编译程序(Compiler):** 高级语言源程序→机器级目标程序
 - 解释程序(Interpreter):** 将高级语言语句逐条翻译成机器指令并立即执行, 不生成目标文件。
 - 其他实用程序: 如: 磁盘碎片整理程序、备份程序等
- **Application software(应用软件)** - 解决具体应用问题/完成具体应用任务
 - 各类媒体处理程序: Word/ Image/ Graphics/...
 - 管理信息系统 (MIS)

Computer Hierarchy (计算机系统层次)

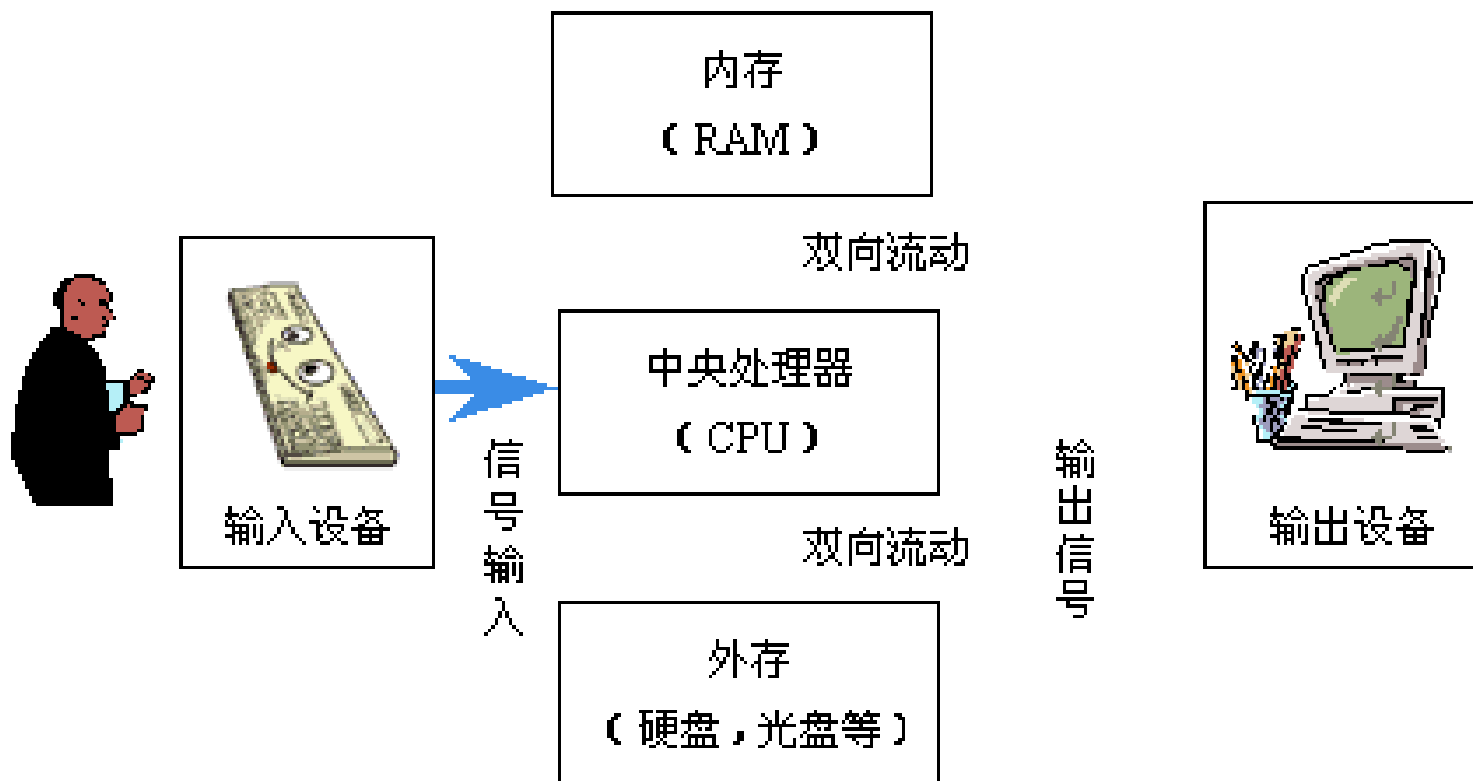
Carnegie Mellon



。上图给出的是计算机系统的层次结构

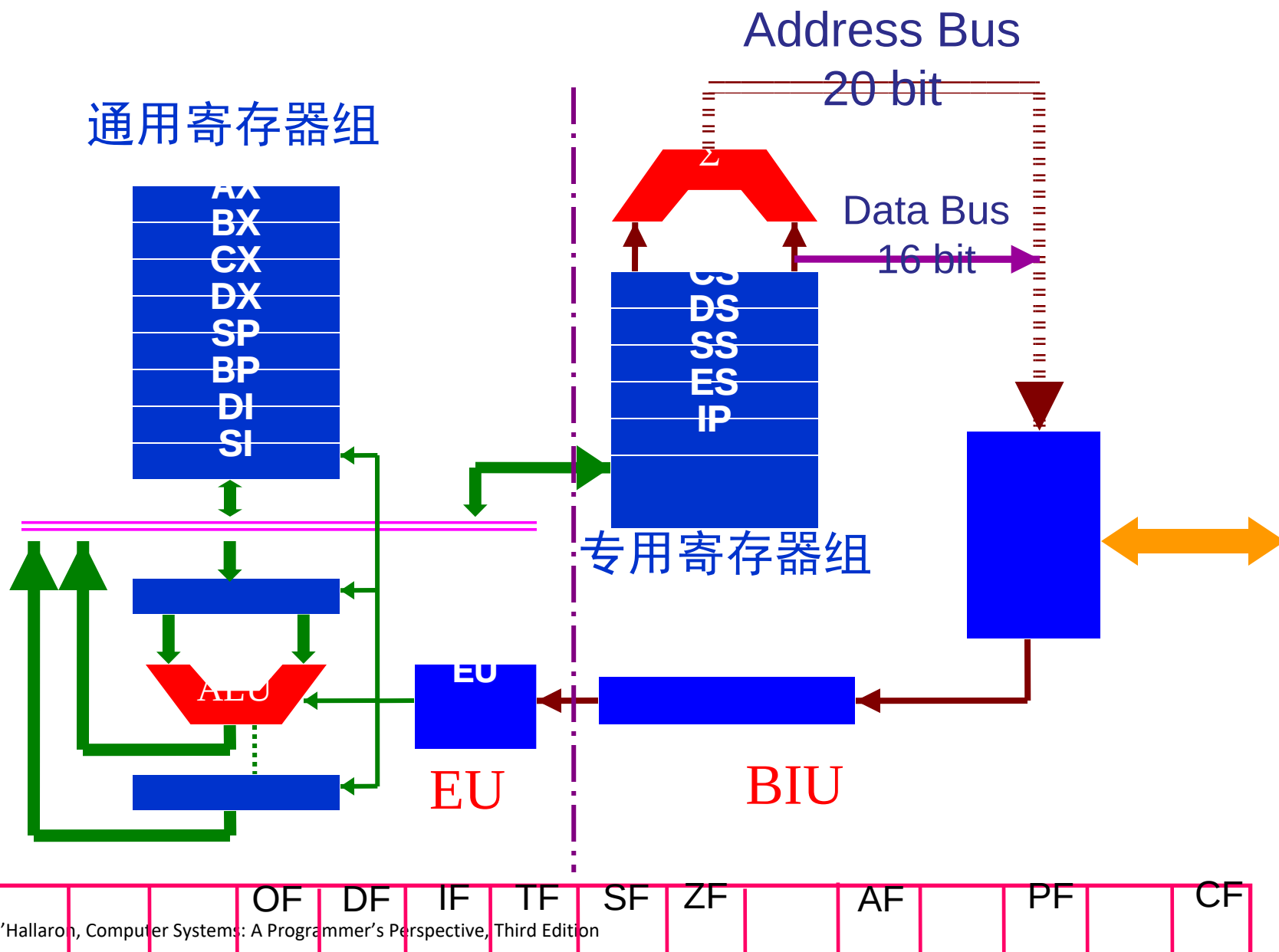
指令系统（即ISA）是软/硬件的交界面

。不同用户工作在不同层次，所看到的计算机不一样



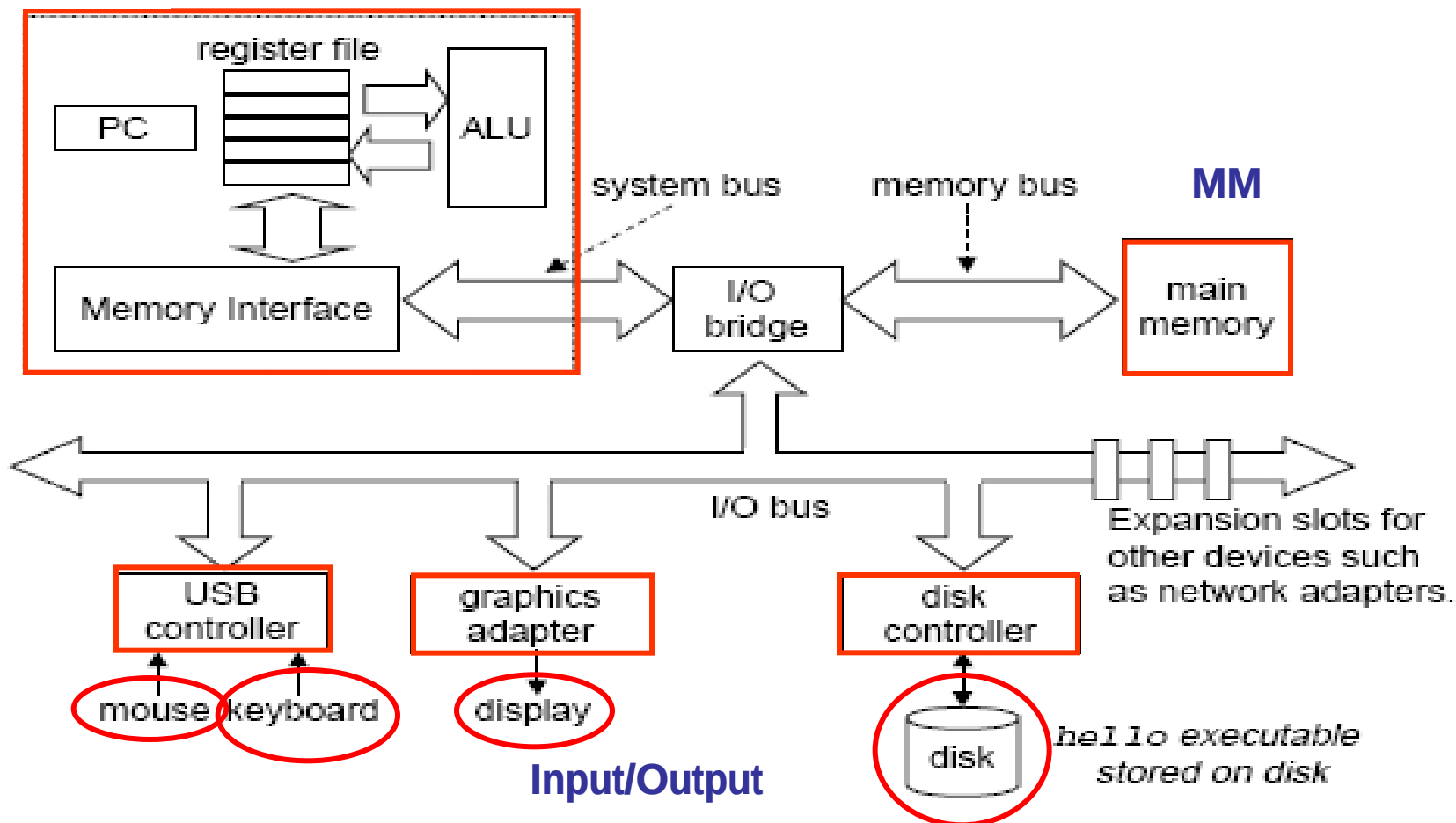
“存储程序”的思想确立了现代计算机的结构

8086CPU内部结构框图



一个典型系统的硬件组成

CPU



PC 程序计数器； **ALU**：算术/逻辑单元； **USB**：通用串行总线

一个典型程序的转换处理过程

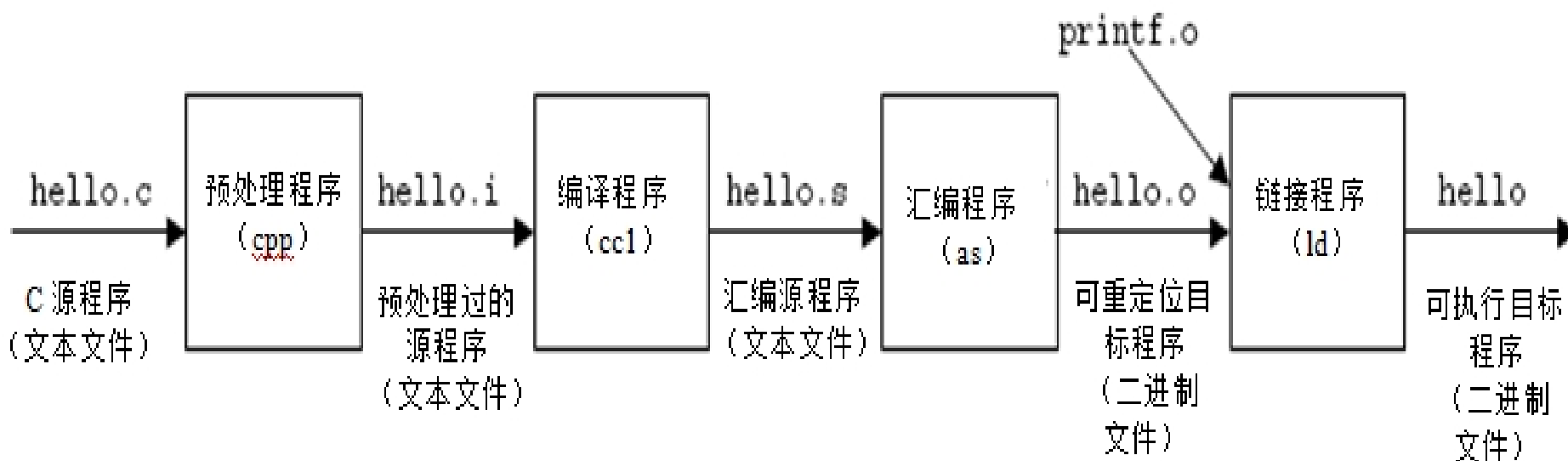
经典的“hello.c”C-源程序

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("hello, world\n");
6 }
```

程序的功能是：
输出“hello,world”

hello.c的ASCII文本表示

```
#include <stdio.h>
35 105 110 99 108 117 100 101 32 60 115 116 100 105 111 46
h > \n \n int main() \n {
104 62 10 10 105 110 116 32 109 97 105 110 40 41 10 123
\n <sp> <sp> <sp> <sp> printf( " hel
10 32 32 32 32 112 114 105 110 116 102 40 34 104 101 108
lo, <sp> world\n " ); \n }
108 111 44 32 119 111 114 108 100 92 110 34 41 59 10 125
```



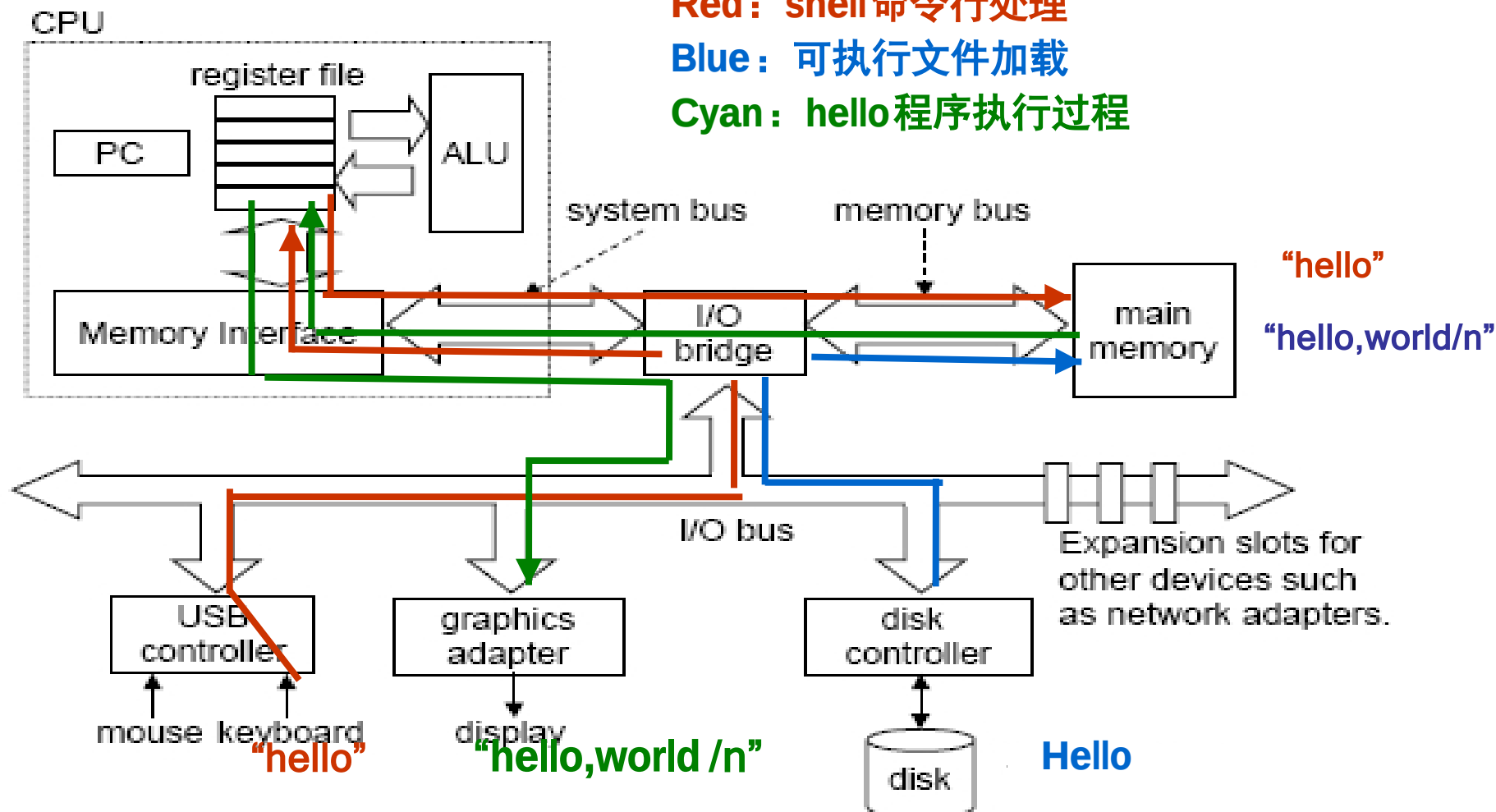
Hello程序的数据流动过程

```
Unix> ./hello [Enter]
hello, world
unix>
```

Red: shell 命令行处理

Blue: 可执行文件加载

Cyan: hello 程序执行过程



数据经常在各存储部件间传送。故现代计算机大多采用“缓存”技术！
所有过程都是在CPU执行指令所产生的控制信号的作用下进行的。